



**INSTITUTO DE EDUCACIÓN SECUNDARIA LA MOLA
NOVELDA**

DESARROLLO DE APLICACIONES WEB

Curso Académico 2025/2026

Desarrollo Web en Entorno Cliente

**U03_A04 Casos prácticos, Ponte a prueba, etc... -
Prof. Emilio Ramírez**

Autor: Segura Abad, Mario

Fecha: 25 de Noviembre, 2025

ÍNDICE

INTRODUCCIÓN..... 3

ENUNCIADO 4

 PROBAD TODO EL CÓDIGO DEL APARTADO 44

 PROBAD LOS SCRIPTS DEL APARTADO 47

 CASO PRÁCTICO 4.....10

 Objetos manejadores de eventos: versión 2.....10

 PONTE A PRUEBA 412

 Contador con valor de inicio personalizado12

 CLAVES DE RESOLUCIÓN:.....14

DESARROLLO DE LA PRACTICA 15

 DESCRIPCIÓN15

 CONTENIDO DEL ARCHIVO.....15

 Contenido del Apartado 4 de teoría.....15

 Caso Práctico 4 – Objetos manejadores de eventos: versión 2.....18

 Ponte a Prueba 4 – Contador con valor de inicio personalizado.....19

 CÓMO VISUALIZARLO20

 Requisitos:20

 Pasos:.....20

 OBJETIVOS DE LA PRÁCTICA.....20

 • Probar y comprender el código teórico del apartado 4 de JavaScript.....20

 • Crear componentes web personalizados con Shadow DOM.20

 • Implementar atributos para inicializar valores en componentes.20

 • Documentar correctamente cada ejercicio, **como siempre**, con nombre, fecha y capturas de pantalla.....20

 • Presentar el proyecto en formato PDF junto con los archivos fuente y pantallazos.20

CONCLUSIÓN..... 21

INTRODUCCIÓN

En el ejercicio “Probad todo el código del Apartado 4” deberemos incluir en uno o varios fuentes (siempre recomendando varios) absolutamente todo el código del apartado 4 de la teoría de JavaScript, así como su ejecución y prueba de autoría.

En el ejercicio “Probad los Scripts del Apartado 5” deberemos ejecutar los scripts adjuntos en el ejercicio y explicaremos su funcionamiento, asimismo realizaremos sus correspondientes pruebas de autoría.

En el “Caso práctico 4” vamos a resolver de nuevo el Caso práctico 2, pero esta vez utilizando algún componente web. Disponemos del archivo DWEC_U03_A02_CP_E.js con un array de “chistes”. Crearemos un programa en JS que muestre para cada chiste un elemento <mi-chiste> conteniendo así el enunciado y la respuesta del chiste.

En el “Ponte a prueba 4” modificaremos el ejemplo de contador completo en la teoría para que permita definir un valor inicial a través de un atributo.

ENUNCIADO

Probad todo el código del Apartado 4

Incluid en uno o varios fuentes **todo** el código que aparece en la teoría de este apartado, así como su ejecución y prueba de autoría.

Ejemplo de implementación:

Captura de pantalla – DWEC_U03_A04_TEO_1.js

```
JS DWEC_U03_A04_TEO_1.js X
JS DWEC_U03_A04_TEO_1.js > MiElemento
1  // Autor: Mario Segura Abad
2  // Fecha: 23/11/2025
3
4
5  // 4.2. Shadow DOM
6  class MiElemento extends HTMLElement {
7
8      constructor() {
9          super();
10
11          // Inicializamos la cuenta
12          this.cuenta = 0;
13      }
14
15      connectedCallback() {
16          // Añadimos el Shadow DOM al elemento
17          const shadow = this.attachShadow({mode: 'open'});
18
19          let boton = document.createElement("button");
20          boton.textContent = this.cuenta;
21          boton.addEventListener("click", () => {
22              this.contar();
23              boton.textContent = this.cuenta;
24          });
25
26          // Añadimos el botón al Shadow DOM del elemento, en lugar de
27          // añadirlo al elemento directamente
28          shadow.append(boton);
29      }
30
31      contar() {
32          this.cuenta++;
33      }
34  }
35
36  // Registramos el componente personalizado
37  customElements.define("mi-elemento", MiElemento);
```

Captura de pantalla – DWEC_U03_A04_TEO_1.html

```
<> DWEC_U03_A04_TEO_1.html X
<> DWEC_U03_A04_TEO_1.html > ...
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 23/11/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6      <head>
7          <meta charset="UTF-8">
8          <meta name="viewport" content="width=device-width, initial-scale=1.0">
9          <title>4.2. Shadow DOM</title>
10     </head>
11     <body>
12         <h2>Probando Contador con Web Components</h2>
13
14         <!-- Uso del componente -->
15         <mi-elemento></mi-elemento>
16
17         <!-- Importación del archivo JS -->
18         <script src="DWEC_U03_A04_TEO_1.js"></script>
19
20         <!-- Prueba de autoría -->
21         <br><p>Autor: <strong>Mario Segura Abad</strong></p>
22         <p>Fecha de realización: 23/11/2025</p>
23     </body>
24 </html>
```

Captura de pantalla – DWEC_U03_A04_TEO_2.js

```
JS DWEC_U03_A04_TEO_2.js X
JS DWEC_U03_A04_TEO_2.js > ...
1  // Autor: Mario Segura Abad
2  // Fecha: 23/11/2025
3
4
5  // 4.2. Shadow DOM
6  document.getElementById("p1"); // Devuelve el primer párrafo
7  document.getElementById("p4"); // undefined (está en el Shadow DOM)
8
9  let miElemento = document.getElementById("elem1");
10 miElemento.shadowRoot.getElementById("p1"); // Devuelve el párrafo 3 (accedemos a través del Shadow DOM)
11
12 document.querySelectorAll("p"); // Devuelve los dos primeros párrafos
13 miElemento.shadowRoot.querySelectorAll("p"); // Devuelve los dos últimos párrafos
```

Captura de pantalla – DWEC_U03_A04_TEO_2.html

```
<> DWEC_U03_A04_TEO_2.html X
<> DWEC_U03_A04_TEO_2.html > ...
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 23/11/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6    <head>
7      <meta charset="UTF-8">
8      <meta name="viewport" content="width=device-width, initial-scale=1.0">
9      <title>4.2. Shadow DOM</title>
10   </head>
11   <body>
12     <p class="parrafo" id="p1">Párrafo 1</p>
13     <p class="parrafo" id="p2">Párrafo 2</p>
14
15     <mi-elemento id="elem1">
16       <!-- Su contenido está creado en el Shadow DOM en modo 'open' -->
17       <p class="parrafo" id="p1">Párrafo 3</p> <!-- Es válido: 'p1' está aislado del DOM exterior -->
18       <p class="parrafo" id="p4">Párrafo 4</p>
19     </mi-elemento>
20
21     <!-- Prueba de Autoría -->
22     <br><p>Autor: <strong>Mario Segura Abad</strong></p>
23     <p>Fecha de realización: 23/11/2025</p>
24   </body>
25 </html>
```

Captura de pantalla – DWEC_U03_A04_TEO_3.js

```
JS DWEC_U03_A04_TEO_3.js X
JS DWEC_U03_A04_TEO_3.js > ...
1  // Autor: Mario Segura Abad
2  // Fecha: 23/11/2025
3
4
5  // 4.3. Templates
6  let plantilla = document.getElementById('mi-boton'); // Elemento <template>
7  let plantillaContenido = plantilla.content; // Contenido de la plantilla (nodo tipo 'DocumentFragment')
8  document.body.append(plantillaContenido.cloneNode(true)); // Se añade a <body> una COPIA de la plantilla
9  document.body.append(plantillaContenido.cloneNode(true)); // Se añade a <body> una segunda COPIA de la plantilla
```

Captura de pantalla – DWEC_U03_A04_TEO_3.html

```
<> DWEC_U03_A04_TEO_3.html X
<> DWEC_U03_A04_TEO_3.html > html
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 23/11/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6    <head>
7      <meta charset="UTF-8">
8      <meta name="viewport" content="width=device-width, initial-scale=1.0">
9      <title>4.3. Templates</title>
10   </head>
11   <body>
12     <h1>Abrir la consola (F12 ->Consola), pegar el código .js en dicha consola y pulsar Enter</h1>
13     <template id="mi-boton">
14       <button>Mi Botón</button>
15     </template>
16
17     <!-- Prueba de Autoría -->
18     <br><p>Autor: <strong>Mario Segura Abad</strong></p>
19     <p>Fecha de realización: 23/11/2025</p>
20   </body>
21 </html>
```

Captura de pantalla – DWEC_U03_A04_TEO_4.html

```
<> DWEC_U03_A04_TEO_4.html X
<> DWEC_U03_A04_TEO_4.html > ...
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 24/11/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6      <head>
7          <meta charset="UTF-8">
8          <meta name="viewport" content="width=device-width, initial-scale=1.0">
9          <title>4.4. Slots</title>
10     </head>
11     <body>
12         <mi-contador>
13             <span>Texto personalizado</span>
14         </mi-contador>
15
16         <!-- Suponiendo que se instancia en un Componente personalizado 'mi-componente' -->
17         <template>
18             <div>
19                 <slot name="texto">Texto por defecto</slot>
20             </div>
21         </template>
22
23         <mi-componente>
24             <br><br><span slot="texto">Juan</span>
25         </mi-componente>
26
27         <mi-componente>
28             <p slot="texto">Laura</p>
29         </mi-componente>
30
31         <mi-componente>
32         </mi-componente>
33
34         <!-- Prueba de autoría -->
35         <br><br><p>Autor: <strong>Mario Segura Abad</strong></p>
36         <p>Fecha de realización: 24/11/2025</p>
37     </body>
38 </html>
```

Probad los Scripts del Apartado 4

Ejecutad los scripts adjuntos y explicad su funcionamiento. Recordad incluir pruebas de autoría.

Ejemplo de implementación:

Captura de pantalla – DWECE_U03_A04_01.html

```
<> DWECE_U03_A04_01.html X
Scripts Apartado 4 > <> DWECE_U03_A04_01.html > ...
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 24/11/2025 -->
3
4  <script>
5      // Clase que define el comportamiento del contador
6      class MiContador extends HTMLElement {
7
8          // El constructor se ejecuta cuando se crea el elemento
9          // Importante: el elemento aún no está en el DOM, por lo que no
10         // podemos hacer referencia a las funciones del DOM
11         // (aunque sí podremos usar el Shadow DOM, que veremos más adelante)
12         constructor() {
13             // Lo primero que hay que hacer es ejecutar 'super()' para
14             // llamar al constructor del prototipo HTMLElement
15             super();
16             // Inicializamos la propiedad 'cuenta'
17             this.cuenta = 0;
18         }
19
20         // Función de cuenta
21         contar() {
22             this.cuenta++;
23         }
24
25         // Esta función se ejecuta cuando se añade el elemento al DOM
26         // Por tanto, podremos hacer referencia a las funciones del DOM,
27         // como 'innerHTML', 'createElement', etcétera.
28         connectedCallback() {
29             // Creamos un botón
30             let boton = document.createElement("button");
31             // que muestra el valor actual de 'cuenta'
32             boton.textContent = this.cuenta;
33             // Manejador de evento 'click'
34             // Función flecha: si se usara una función convencional, la función se ejecutaría
35             // sin contexto de objeto, por lo que 'this' apuntaría al objeto global
36             boton.addEventListener("click", () => {
37                 // Cada vez que se pulsa, se llama a la función 'contar'
38                 this.contar()
39                 // Se actualiza el valor del botón
40                 boton.textContent = this.cuenta;
41             })
42         }
43     }
44
45     // Registrar la clase en el navegador
46     customElements.define("mi-contador", MiContador);
47
48     // Crear un elemento de la clase
49     const contador = document.createElement("mi-contador");
50     // Añadir el elemento al DOM
51     document.body.appendChild(contador);
```


Captura de pantalla – DWEC_U03_A04_02.html

```
<> DWEC_U03_A04_02.html X
Scripts Apartado 4 > <> DWEC_U03_A04_02.html > br
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 24/11/2025 -->
3
4  <template id="mi-contador">
5    <style>
6      button {
7        padding: 1em;
8        border-radius: 1em;
9      }
10   </style>
11   <button></button>
12 </template>
13
14 <script>
15   // Clase que define el comportamiento del contador
16   class MiContador extends HTMLElement {
17     constructor() {
18       super();
19       this.cuenta = 0;
20     }
21
22     // Función de cuenta
23     contar() {
24       this.cuenta++;
25     }
26
27     // Vamos a utilizar Shadow DOM, así que podríamos poner esta
28     // funcionalidad también en el constructor
29     connectedCallback() {
30       // Creamos el Shadow DOM
31       const shadow = this.attachShadow({mode: 'open'});
32
33       // Cargamos la plantilla
34       let plantilla = document.getElementById('mi-contador'); // Elemento <template>
35       let plantillaContenido = plantilla.content; // Contenido de la plantilla (nodo tipo 'DocumentFragment')
36
37       // Se añade al Shadow DOM una COPIA de la plantilla
38       shadow.append(plantillaContenido.cloneNode(true));
39
40       // Buscamos el botón dentro del Shadow DOM
41       let boton = shadow.querySelector("button");
42       boton.textContent = this.cuenta; // Valor de 'cuenta'
```

Captura de pantalla – DWEC_U03_A04_03.html

```
<> DWEC_U03_A04_03.html X
Scripts Apartado 4 > <> DWEC_U03_A04_03.html > ...
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 24/11/2025 -->
3
4  <template id="mi-contador">
5    <style>
6      #contenedor {
7        padding: 1em;
8        border: 1px solid black;
9        display: inline-block;
10     }
11     button {
12       padding: 1em;
13       border-radius: 1em;
14       display: block;
15       margin: auto;
16     }
17   </style>
18   <div id="contenedor">
19     <!-- Slot -->
20     <slot name="titulo">Título por defecto</slot>
21     <button></button>
22   </div>
23 </template>
24
25 <script>
26   // Clase que define el comportamiento del contador
27   class MiContador extends HTMLElement {
28     constructor() {
29       super();
30       this.cuenta = 0;
31     }
32
33     // Función de cuenta
34     contar() {
35       this.cuenta++;
36     }
37
38     // Vamos a utilizar Shadow DOM, así que podríamos poner esta
39     // funcionalidad también en el constructor
40     connectedCallback() {
41       // Creamos el Shadow DOM
42       const shadow = this.attachShadow({mode: 'open'});
```

Caso práctico 4

Objetos manejadores de eventos: versión 2

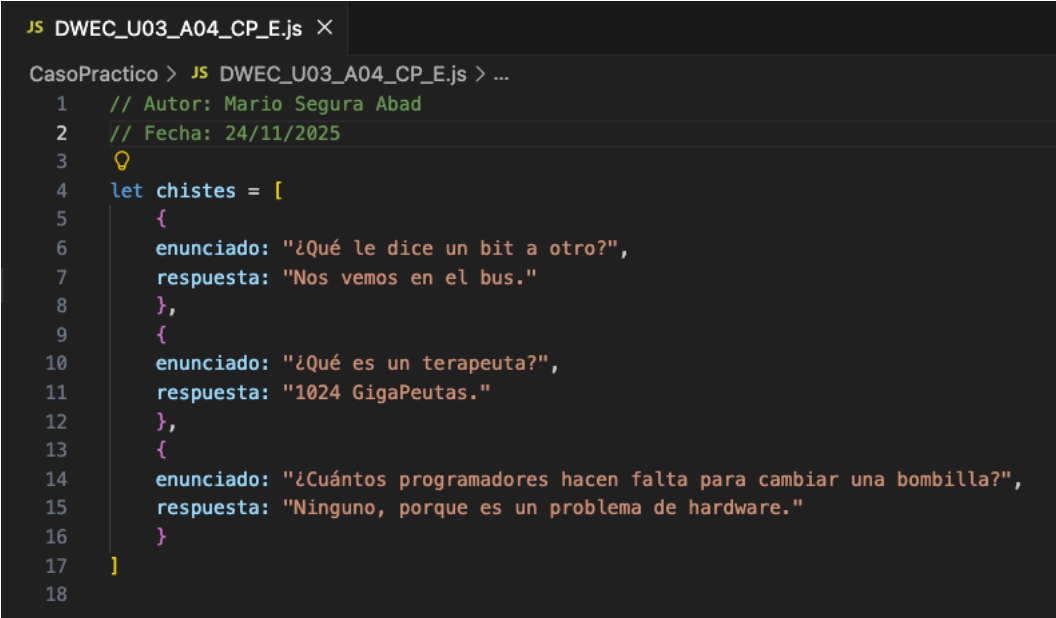
En este caso práctico vamos a resolver de nuevo el mismo supuesto del Caso práctico 2, pero esta vez utilizando componentes web. Adjunto dispones del archivo **DWEC_U03_A02_CP_E.js**, con un *array* de objetos “chiste”.

Crea un programa en JavaScript que muestre para cada chiste un elemento **<mi-chiste>** que contenga el enunciado y un botón con el texto “Ver respuesta”. Al pulsar este botón, se debe mostrar mediante un **alert** la respuesta correspondiente. Debes utilizar:

- Una plantilla HTML para crear la estructura formada por el enunciado y el botón.
- Un componente personalizado que se encargue de realizar las siguientes tareas:
- Cargar la plantilla.
- Añadirla al componente usando el Shadow DOM.
- Añadir el texto del enunciado y el manejador de eventos del botón.
Los datos del objeto chiste se pasarán en una propiedad del elemento en el momento de su creación.

Ejemplo de implementación:

Captura de pantalla – DWEC_U03_A04_CP_E.js



```
JS DWEC_U03_A04_CP_E.js X
CasoPractico > JS DWEC_U03_A04_CP_E.js > ...
1 // Autor: Mario Segura Abad
2 // Fecha: 24/11/2025
3
4 let chistes = [
5   {
6     enunciado: "¿Qué le dice un bit a otro?",
7     respuesta: "Nos vemos en el bus."
8   },
9   {
10    enunciado: "¿Qué es un terapeuta?",
11    respuesta: "1024 GigaPeutas."
12  },
13  {
14    enunciado: "¿Cuántos programadores hacen falta para cambiar una bombilla?",
15    respuesta: "Ninguno, porque es un problema de hardware."
16  }
17 ]
18
```

Captura de pantalla – DWEC_U03_A04_CP_E_COMPONENT.js

```
JS DWEC_U03_A04_CP_E_COMPONENT.js x
CasoPractico > JS DWEC_U03_A04_CP_E_COMPONENT.js > MiChiste > connectedCallback > contenido
1 // Autor: Mario Segura Abad
2 // Fecha: 24/11/2025
3
4 class MiChiste extends HTMLElement {
5   constructor() {
6     super();
7
8     this.datos = null; // aquí se guardará el objeto {enunciado, respuesta}
9
10    this.attachShadow({ mode: "open" });
11  }
12
13  connectedCallback() {
14    const tpl = document.getElementById("tpl-chiste");
15    const contenido = tpl.content.cloneNode(true);
16
17    // Insertar en el Shadow DOM
18    this.shadowRoot.appendChild(contenido);
19
20    // Añadir enunciado
21    this.shadowRoot.querySelector(".enunciado").textContent = this.datos.enunciado;
22
23    // Botón
24    const boton = this.shadowRoot.querySelector("button");
25
26    // Evento
27    boton.addEventListener("click", () => {
28      alert(this.datos.respuesta);
29    });
30  }
31
32
33  customElements.define("mi-chiste", MiChiste);
```

Captura de pantalla – CasoPractico4.html

```

<> CasoPractico4.html X
CasoPractico > <> CasoPractico4.html > html > body > br
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 24/11/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="en">
6      <head>
7          <meta charset="UTF-8">
8          <meta name="viewport" content="width=device-width, initial-scale=1.0">
9          <title>Caso Práctico 4 - Web Components</title>
10     </head>
11     <body>
12         <h2>Chistes con Web Components</h2>
13
14         <!-- Plantilla -->
15         <template id="tpl-chiste">
16             <div class="chiste">
17                 <p class="enunciado"></p>
18                 <button>Ver respuesta</button>
19             </div>
20         </template>
21
22         <!-- Aquí aparecerán los chistes -->
23         <div id="contenedor"></div>
24
25         <!-- Archivo con el array de chistes -->
26         <script src="DWE_C_U03_A04_CP_E.js"></script>
27
28         <!-- Componente -->
29         <script src="DWE_C_U03_A04_CP_E_COMPONENT.js"></script>
30
31         <!-- Creación de los elementos -->
32         <script>
33             chistes.forEach(chiste => {
34                 const elem = document.createElement("mi-chiste");
35                 elem.datos = chiste; // pasamos datos al componente
36                 document.getElementById("contenedor").appendChild(elem);
37             });
38         </script>
39
40         <!-- Prueba de autoría -->
41         <br><br><p>Autor: <strong>Mario Segura Abad</strong></p>
42         <p>Fecha de realización: 24/11/2025</p>

```

Ponte a prueba 4

Contador con valor de inicio personalizado

Modifica el ejemplo de contador completo estudiado en el apartado teórico para que permita definir un valor inicial a través de un atributo. A continuación, se muestran algunos ejemplos de utilización:

```

<mi-contador iniciar="5"></mi-contador>
<mi-contador iniciar="10"></mi-contador>
<mi-contador></mi-contador> <!-- Se inicia en 0 -->

```

Ejemplo de implementación: Captura de pantalla – PonteAPrueba4.js

```

JS PonteAPrueba4.js X
PonteAPrueba > JS PonteAPrueba4.js > MiContador > connectedCallback
1 // Autor: Mario Segura Abad
2 // Fecha: 24/11/2025
3
4 class MiContador extends HTMLElement {
5   constructor() {
6     super();
7
8     // Inicializamos Shadow DOM
9     this.attachShadow({ mode: "open" });
10
11     // Valor inicial (se leerá de atributo en connectedCallback)
12     this.cuenta = 0;
13   }
14
15   connectedCallback() {
16     // Leer el atributo "iniciar"
17     const valorInicial = this.getAttribute("iniciar");
18
19     // Si existe, convertirlo a número. Si no, dejar en 0.
20     this.cuenta = valorInicial ? parseInt(valorInicial) : 0;
21
22     // Crear el botón
23     const boton = document.createElement("button");
24     boton.textContent = this.cuenta;
25
26     // Evento de incremento
27     boton.addEventListener("click", () => {
28       this.cuenta++;
29       boton.textContent = this.cuenta;
30     });
31
32     // Añadir al Shadow DOM
33     this.shadowRoot.appendChild(boton);
34   }
35 }
36
37 customElements.define("mi-contador", MiContador);

```

Captura de pantalla – PonteAPrueba4.html

```

PonteAPrueba > PonteAPrueba4.html > html > body > br
1 <!-- Autor: Mario Segura Abad -->
2 <!-- Fecha: 24/11/2025 -->
3
4 <!DOCTYPE html>
5 <html lang="en">
6   <head>
7     <meta charset="UTF-8">
8     <meta name="viewport" content="width=device-width, initial-scale=1.0">
9     <title>Ponte a Prueba 4</title>
10  </head>
11  <body>
12    <h2>Contadores Personalizados</h2>
13
14    <mi-contador iniciar="5"></mi-contador>
15    <mi-contador iniciar="10"></mi-contador>
16    <mi-contador></mi-contador> <!-- empieza en 0 -->
17
18    <script src="PonteAPrueba4.js"></script>
19
20    <!-- Prueba de autoría -->
21    <br><br><br><p>Autor: <strong>Mario Segura Abad</strong></p>
22    <p>Fecha de realización: 24/11/2025</p>
23  </body>
24 </html>

```

Claves de resolución:

- Los cambios han de ser realizados dentro de **connectCallback**.
- El elemento personalizado es accesible dentro de dicha función a través de **this**.
- Para obtener el valor de un atributo de un elemento, puedes utilizar la función **getAttribute**.
- Debes tener en cuenta el caso de que el atributo no esté definido.

DESARROLLO DE LA PRACTICA

Práctica DWEC – Unidad 3 Apartado 4

Descripción

Este apartado se centra en el uso de **componentes web personalizados** y el **Shadow DOM** para encapsular lógica y presentación.

El objetivo principal ha sido repetir el ejercicio de los chistes (Caso práctico 2) pero ahora con un componente `<mi-chiste>`, y modificar el ejemplo del contador para que acepte un valor inicial mediante un atributo.

Se ha trabajado con el archivo `DWEC_U03_A02_CP_E.js` y con ejemplos de componentes personalizados en HTML y JavaScript.

Contenido del archivo

Contenido del Apartado 4 de teoría

Se ha probado todo el código relacionado con:

- Creación de **plantillas HTML** para reutilizar estructuras.
- Definición de **componentes personalizados** con `customElements.define`.
- Uso del **Shadow DOM** para encapsular estilos y estructura.
- Lectura de atributos en componentes mediante `getAttribute`.

Cada archivo incluye comentarios con el nombre del autor y la fecha de realización. Se han capturado los resultados por consola y pantalla como prueba de ejecución.

Ejemplo de ejecución:

Salida por consola – `DWEC_U03_A04_TEO_1.html`

Probando Contador con Web Components

3

Autor: **Mario Segura Abad**

Fecha de realización: 23/11/2025

Salida por consola – DWEC_U03_A04_TEO_2.html

Párrafo 1

Párrafo 2

Párrafo 3

Párrafo 4

Autor: **Mario Segura Abad**

Fecha de realización: 23/11/2025

Salida por consola – DWEC_U03_A04_TEO_3.html

Abrir la consola (F12 ->Consola), pegar el código .js en dicha consola y pulsar Enter

Autor: **Mario Segura Abad**

Fecha de realización: 23/11/2025

Fecha de realización: 23/11/2025

Mi Botón

Mi Botón

DevTools is now available in Spanish

Don't show again

Always match Chrome's language

Switch DevTools to Spanish

Elements

Console

Sources

Network

Performance

Memory

Application

Privacy and security

Lighthouse

Rec

top

Filter

> // Autor: Mario Segura Abad

// Fecha: 23/11/2025

// 4.3. Templates

let plantilla = document.getElementById('mi-boton'); // Elemento <template>

let plantillaContenido = plantilla.content; // Contenido de la plantilla (nodo tipo 'DocumentFragment')

document.body.append(plantillaContenido.cloneNode(true)); // Se añade a <body> una COPIA de la plantilla

document.body.append(plantillaContenido.cloneNode(true)); // Se añade a <body> una segunda COPIA de la plantilla

< undefined

> |

Salida por consola – DWEC_U03_A04_TEO_4.html

Texto personalizado

Juan

Laura

Autor: **Mario Segura Abad**

Fecha de realización: 24/11/2025

Salida por consola – DWEC_U03_A04_01.html

1

2

1

Autor: **Mario Segura Abad**

Fecha de realización: 24/11/2025

Captura de pantalla – DWEC_U03_A04_02.html

0

0

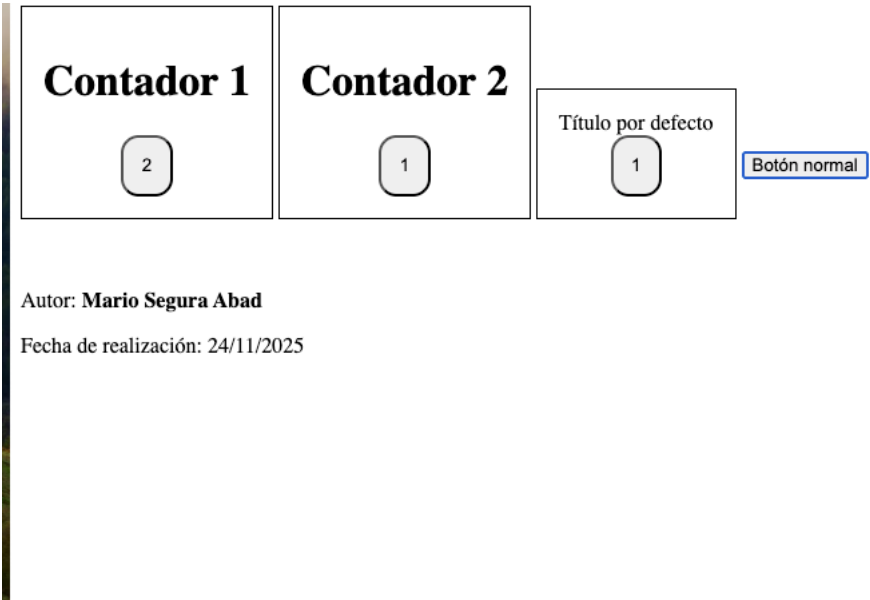
0

Botón normal

Autor: **Mario Segura Abad**

Fecha de realización: 24/11/2025

Captura de pantalla – DWECS_U03_A04_03.html



Caso Práctico 4 – Objetos manejadores de eventos: versión 2

1. Componente <mi-chiste>

- Se ha creado una plantilla HTML con un enunciado y un botón **Ver respuesta**.
- Se ha definido un componente personalizado que:
 - Carga la plantilla en su Shadow DOM.
 - Inserta el texto del enunciado recibido como propiedad.
 - Añade el manejador de eventos al botón para mostrar la respuesta con alert.
- Cada objeto del array de chistes se convierte en un elemento <mi-chiste> en la página.

Este ejercicio ha reforzado el uso de **componentes web** y la encapsulación de lógica y estilos.

Ejemplo de ejecución:

Salida por consola – CasoPracico4.html

Chistes con Web Components

¿Qué le dice un bit a otro?

Ver respuesta

¿Qué es un terapeuta?

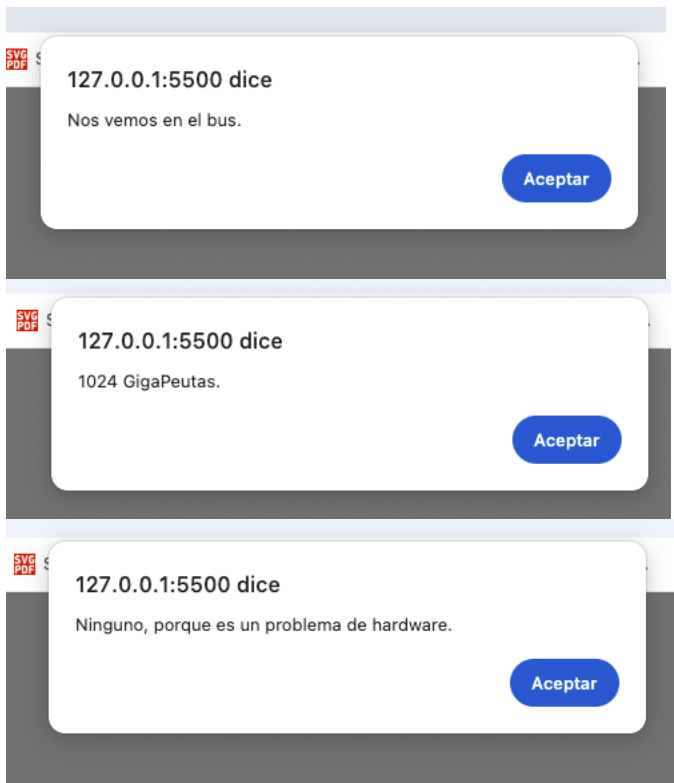
Ver respuesta

¿Cuántos programadores hacen falta para cambiar una bombilla?

Ver respuesta

Autor: **Mario Segura Abad**

Fecha de realización: 24/11/2025



Ponte a Prueba 4 – Contador con valor de inicio personalizado

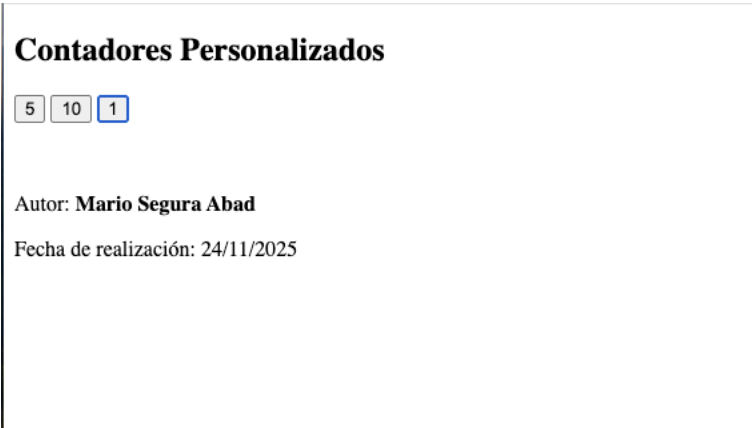
1. Componente <mi-contador>

- Se ha modificado el ejemplo del contador para aceptar un atributo `iniciar`.
- En el `connectedCallback`, se obtiene el valor con `this.getAttribute("iniciar")`.
- Si el atributo no está definido, el contador comienza en 0.

Este ejercicio ha reforzado el uso de **atributos personalizados** y la inicialización de componentes web.

Ejemplo de ejecución:

Salida por consola – PonteAPrueba4.html



Cómo visualizarlo

Requisitos:

- Editor de código compatible: en mi caso, Visual Studio Code.
- Navegador web actualizado: en mi caso, Google Chrome.

Pasos:

1. Guardar los archivos .html y .js en la misma carpeta.
2. Abrir el archivo .html en el navegador.
3. **Caso práctico 4:** comprobar que cada chiste aparece como un componente `<mi-chiste>` y que al pulsar el botón se muestra la respuesta.
4. **Ponte a prueba 4:** insertar varios `<mi-contador>` en el HTML y verificar que cada uno inicia en el valor indicado por su atributo.
5. Revisar la consola del navegador (F12 → Consola) para verificar mensajes de prueba de autoría.

Objetivos de la práctica

- Probar y comprender el código teórico del apartado 4 de JavaScript.
- Crear componentes web personalizados con Shadow DOM.
- Implementar atributos para inicializar valores en componentes.
- Documentar correctamente cada ejercicio, **como siempre**, con nombre, fecha y capturas de pantalla.
- Presentar el proyecto en formato PDF junto con los archivos fuente y pantallazos.

CONCLUSIÓN

Esta práctica nos ha permitido consolidar el uso de **componentes web** en JavaScript, encapsulando estructura y lógica en elementos reutilizables.

Se ha comprobado cómo crear un componente `<mi-chiste>` con eventos propios y cómo modificar un contador para aceptar valores iniciales mediante atributos.

La práctica nos ha sido clave para entender cómo poder construir interfaces modulares y reutilizables en proyectos web futuros.